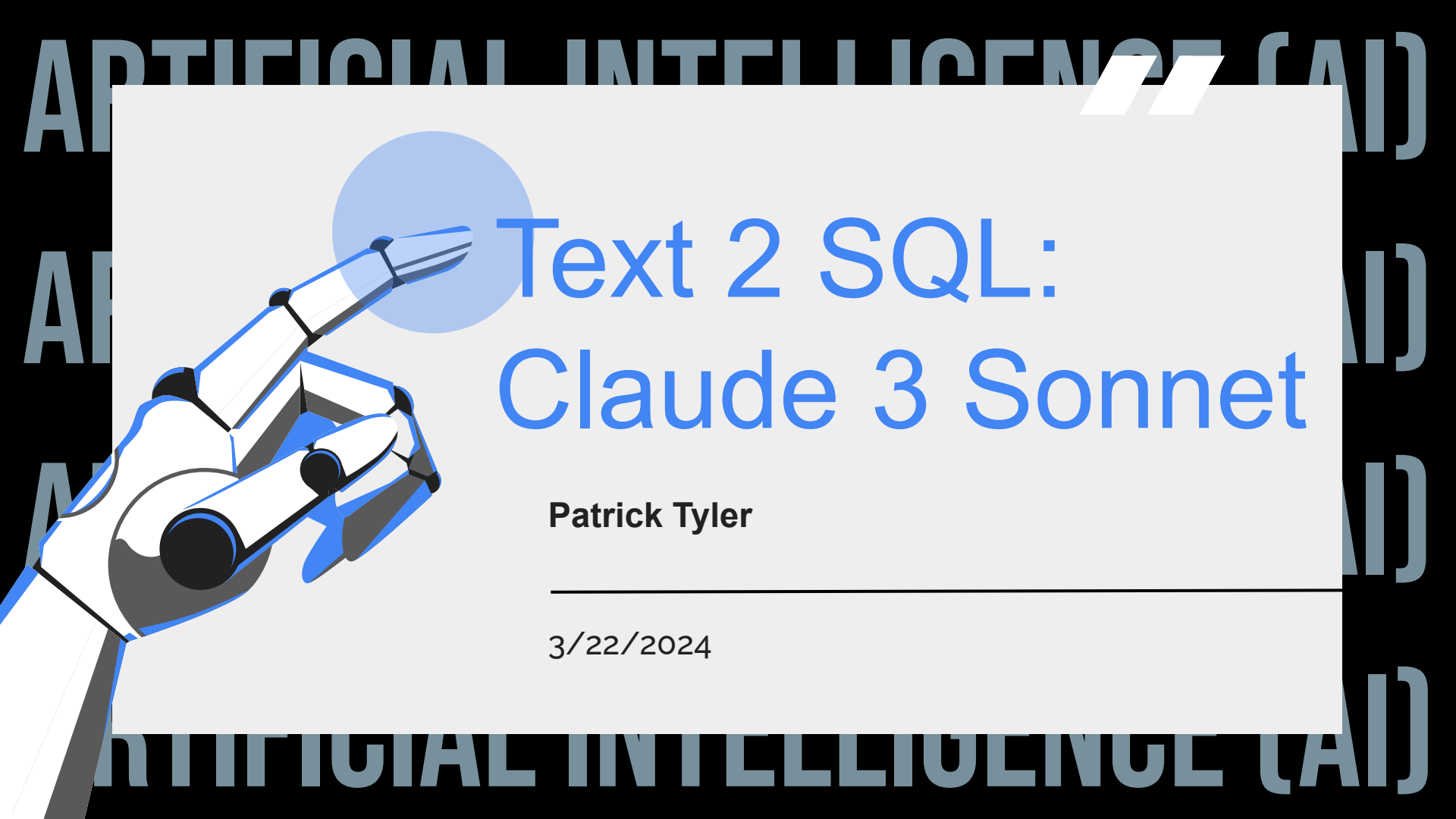




Text 2 SQL: Claude 3 Sonnet

Patrick Tyler

3/22/2024

GRADING METHODOLOGY

7 queries scoring a maximum of 1 point each

- **1 point:** got a fully correct query AND output without help
- **$\frac{3}{4}$ points:** correct query without help
- **$\frac{1}{2}$ points:** correct query with some help
- **$\frac{1}{4}$ points:** correct query with help and examples
- **0 points:** complete nonsense

*my discretion can also be used

<<<<

EXPERIMENT 0 - BASIC UNDERSTANDING¹



- Prompted Claude with the testing script
- Asked Claude about the database and relationships among tables
- Claude correctly identified
 - all tables and columns
 - all relationships
 - Conceptual concepts of how the tables interact with each other

EXPERIMENT - QUERIES



1. Display the cities that makes the most different kinds of products. Experiment with the rank() function.²

- **Correct query answer** interestingly using a view and an **almost** correct query output (9/10 points)
- Claude incorrectly dealt with rank ties and did not increment the next rank

2. Display the names of products whose priceUSD is less than 1% of the average priceUSD, in alphabetical order. from A to Z.³

- **Correct query answer** and **wrong** query output (¾ points)
- Expected since this is a math heavy query and LLM's would need to employ other tools to exact calculations

EXPERIMENT - QUERIES



3. Display the customer last name, product id ordered, and the totalUSD for all orders made in March of any year, sorted by totalUSD from low to high.⁴

- **Correct query answer** and **terrible** query output ($\frac{3}{4}$ points)
- Claude, as expected, cannot do simple math (summation) and also included someone from the wrong month
- Claude also made one letter query abbreviations

4. Display the last name of all customers (in reverse alphabetical order) and their total ordered by customer, and nothing more. Use coalesce to avoid showing NULL totals.⁵

- **Correct query answer** and expected **wrong** query output ($\frac{3}{4}$ points)
- Claude was not just wrong with the totals but also put people name that are not customers
- Interestingly, the person Claude chose for having 0 total orders used is not a customer or an agent

EXPERIMENT - QUERIES



5. Display the names of all customers who bought products from agents based in Chilliwack along with the names of the products they ordered, and the names of the agents who sold it to them.⁶

- **Correct query answer** and **correct** query output (5/4 points)
- Extra credit for concatenating first and last to get customer name

6. Write a query to check the accuracy of the totalUSD column in the Orders table. This means calculating Orders.totalUSD from data in other tables and comparing those values to the values in Orders.totalUSD. Display all rows in Orders where Orders.totalUSD is incorrect, if any. If there are any incorrect values, explain why they are wrong. Round to exactly two decimal places.⁷

- **Correct query answer** and expected **wrong** query output (1 point)
- Extra credit because this query was difficult and again expecting llm's to math is unreasonable
- Surprisingly came up with the correct mathematical rule

EXPERIMENT - QUERIES



7. Display the first and last name of all customers who are also agents.⁸

- **Correct query answer** and **correct** query output (1 point)
- Unsuprisley got smashed this prompt

THOUGHTS AND CONCLUSIONS



- Final grade: 91.4% A great job!
- Claude is impressive and succeeded everywhere a large language model should
- It's query and output format was professional
- It never needed more prompting to get understand the question and generate a query
- Math is (as normal to llm's) its weakness and could be fixed by giving it access to api's that do math (sort of how gpt 4 has access to execute python)
- Although I have not tested other llm's myself, from personal use of bard and gpt Claude it seems, at least preliminarily, better at coding and related topics

RESOURCE #1

Full setup script

```
DROP VIEW IF EXISTS  
PeopleCustomers; DROP  
VIEW IF EXISTS  
PeopleAgents; DROP  
TABLE IF EXISTS Orders;  
DROP TABLE IF EXISTS  
Products; DROP TABLE IF  
EXISTS Customers; DROP
```

PASTED

P Here is the setup script for our postgres database use this as context for the following questions:

RESOURCE #1



What are the tables and their relationships to each other using database terminology

Based on the provided SQL script, the database schema consists of the following tables and their relationships:

1. People Table:

- This is a parent table that stores basic information about people, such as their ID (`pid`), name (prefix, firstName, lastName, suffix), home city, and date of birth.

2. Customers Table:

- This table has a one-to-one relationship with the `People` table, where each customer is associated with a person's record in the `People` table through the `pid` foreign key.
- It stores additional customer-specific information like payment terms and discount percentage.

3. Agents Table:

- This table also has a one-to-one relationship with the `People` table, where each agent is associated with a person's record in the `People` table through the `pid` foreign key.
- It stores agent-specific information like payment terms and commission percentage.

4. Products Table:

- This is an independent table that stores information about products, such as product ID, name, city, quantity on hand, and price.

5. Orders Table:

- This table has a many-to-one relationship with the `Customers` table through the `custId` foreign key, meaning that a customer can have multiple orders.
- It also has a many-to-one relationship with the `Agents` table through the `agentId` foreign key, indicating that an agent can handle multiple orders.
- Additionally, it has a many-to-one relationship with the `Products` table through the `prodId` foreign key, where each order is associated with a specific product.
- This table stores order-related information such as order number, date ordered, customer ID, agent ID, product ID, quantity ordered, and total price.

In summary, the `People` table acts as a central entity, with the `Customers` and `Agents` tables having a one-to-one relationship with it. The `Orders` table serves as a junction table, linking customers, agents, and products through foreign key relationships.

RESOURCE #2

P Write a query to:

Display the cities that makes the most different kinds of products. Experiment with the `rank()` function.

To display the cities that make the most different kinds of products, we can use the `RANK()` function in combination with a subquery and window functions. Here's the query:

sql Copy code

```
WITH product_counts AS (  
  SELECT city, COUNT(DISTINCT prodId) AS product_count  
  FROM Products  
  GROUP BY city  
)  
SELECT city, product_count, RANK() OVER (ORDER BY product_count DESC) AS rank  
FROM product_counts  
ORDER BY rank;
```

Here's how the query works:

1. The `WITH` clause creates a common table expression (CTE) named `product_counts`.

This CTE performs the following operations:

- It selects the `city` and counts the distinct `prodId` values for each city using `COUNT(DISTINCT prodId)`.
- The distinct count of `prodId` represents the number of different kinds of products made in each city.
- The `GROUP BY` clause groups the results by `city`.

2. The main query then selects the `city`, `product_count` (the distinct count of products), and calculates the `RANK()` of each city based on the `product_count` in descending order.
3. The `RANK()` function assigns a rank to each row based on the specified order (`ORDER BY product_count DESC`). Rows with the same value in the `ORDER BY` clause will receive the same rank.
4. The `OVER` clause specifies that the `RANK()` function should be calculated over the entire result set.
5. Finally, the main query orders the results by the `rank` column, effectively displaying the cities with the most different kinds of products first.

This query will show the cities ranked based on the number of distinct products they make, with the city making the most different kinds of products appearing first.

⚠ Claude does not have the ability to run the code it generates yet.

RESOURCE #2

P What would be the output on the sample data that was inserted into the database for that query

Based on the sample data provided in the SQL script, the output of the query to display the cities that make the most different kinds of products, along with their ranking, would be:

Copy code

city	product_count	rank
Newark	4	1
Dallas	2	2
Duluth	2	2
Montreal	1	3
Beijing	1	3

Explanation:

- Newark has 4 distinct products (`p02`, `p07`, `p08`), so it ranks first.
- Dallas and Duluth are tied with 2 distinct products each (`p01`, `p05` for Dallas and `p03`, `p04` for Duluth), so they both get rank 2.
- Montreal and Beijing have 1 distinct product each (`p09` and `p06`, respectively), so they are tied for rank 3.

The `RANK()` function assigns equal ranks to rows with the same value in the `ORDER BY` clause, and the next rank is skipped. In this case, there is no rank 4 because Montreal and Beijing are tied for rank 3.

This output shows that Newark makes the most different kinds of products based on the sample data, followed by Dallas and Duluth, which are tied for the second rank, and then Montreal and Beijing, which are tied for the third rank.

⚠ Claude does not have the ability to run the code it generates yet.

RESOURCE #3

P

Write a query to:

Display the names of products whose priceUSD is less than 1% of the average priceUSD, in alphabetical order. from A to Z.

Also provide what the output of the query would be on the sample data

To display the names of products whose priceUSD is less than 1% of the average priceUSD, in alphabetical order from A to Z, we can use the following query:

```
sql
SELECT name
FROM Products
WHERE priceUSD < (SELECT AVG(priceUSD) * 0.01 FROM Products)
ORDER BY name;
```

Copy code

Here's how the query works:

1. The subquery `SELECT AVG(priceUSD) * 0.01 FROM Products` calculates 1% of the average `priceUSD` across all products.
2. The main query selects the `name` column from the `Products` table.
3. The `WHERE` clause filters the rows where `priceUSD` is less than the calculated value from the subquery (1% of the average `priceUSD`).
4. The `ORDER BY name` clause sorts the results alphabetically by the `name` column.

On the sample data provided, the output of this query would be:

On the sample data provided, the output of this query would be:

Copy code

```
name
-----
Flux Capacitor
HAL 9000 memory chip
```

This is because the `priceUSD` for 'Flux Capacitor' (\$1.00) and 'HAL 9000 memory chip' (\$1.25) are less than 1% of the average `priceUSD` across all products in the sample data.

*context was reset and question was combine for usage purposes and did not have an effect on output

RESOURCE #4

P Write a query to:

Display the customer last name, product id ordered, and the totalUSD for all orders made in March of any year, sorted by totalUSD from low to high.

Also provide what the output of the query would be on the sample data

To display the customer's last name, product ID ordered, and the totalUSD for all orders made in March of any year, sorted by totalUSD from low to high, we can use the following query:

```
sql Copy code
SELECT p.lastName, o.prodId, o.totalUSD
FROM Orders o
JOIN Customers c ON o.custId = c.pid
JOIN People p ON c.pid = p.pid
WHERE EXTRACT(MONTH FROM o.dateOrdered) = 3
ORDER BY o.totalUSD;
```

Here's how the query works:

1. The `FROM` clause starts with the `Orders` table.
2. The `JOIN` clauses join the `Customers` and `People` tables to the `Orders` table to access the customer's last name.
3. The `WHERE` clause filters the rows where the month extracted from the `dateOrdered` column in the `Orders` table is 3 (March).
4. The `SELECT` statement selects the `lastName` from the `People` table, the `prodId` from the `Orders` table, and the `totalUSD` from the `Orders` table.
5. The `ORDER BY o.totalUSD` clause sorts the results by the `totalUSD` column in ascending order.

On the sample data provided, the output of this query would be:

```
Copy code
lastName | prodId | totalUSD
-----+-----+-----
Davis    | p01    | 60977.22
Armstrong | p01    | 56671.55
Ferguson | p06    | 31236.48
Ferguson | p05    | 6550.98
```

This output shows the orders made in March (based on the provided sample data), displaying the customer's last name, the product ID ordered, and the totalUSD for each order, sorted by totalUSD from low to high.

⚠ Claude does not have the ability to run the code it generates yet.

RESOURCE #5

P

Write a query to:

Display the last name of all customers (in reverse alphabetical order) and their total ordered by customer, and nothing more. Use coalesce to avoid showing NULL totals.

Also provide what the output of the query would be on the sample data

To display the last name of all customers in reverse alphabetical order, their total ordered by customer, and avoid showing NULL totals using COALESCE, we can use the following query:

sql

Copy code

```
SELECT p.lastName,  
       COALESCE(SUM(o.totalUSD), 0) AS total_ordered  
FROM Customers c  
JOIN People p ON c.pid = p.pid  
LEFT JOIN Orders o ON c.pid = o.custId  
GROUP BY p.lastName  
ORDER BY p.lastName DESC;
```

Here's how the query works:

1. The `FROM` clause starts with the `Customers` table.
2. The `JOIN` clause joins the `People` table to the `Customers` table to access the customer's last name.
3. The `LEFT JOIN` clause performs a left outer join with the `Orders` table to include customers who have not placed any orders.
4. The `SELECT` statement selects the `lastName` from the `People` table and creates a calculated column `total_ordered` using `COALESCE(SUM(o.totalUSD), 0)`. This adds up the `totalUSD` from the `Orders` table for each customer, and if the sum is NULL (due to no orders for a customer), it replaces the NULL with 0.
5. The `GROUP BY` clause groups the results by the `lastName` column.
6. The `ORDER BY` clause sorts the results by the `lastName` column in descending order.

On the sample data provided, the output of this query would be:

Copy code

lastName	total_ordered
Morrison	48076.40
Gillespie	0.00
Ferguson	233000.02
Davis	76096.81
Armstrong	72442.75

This output shows the last name of all customers in reverse alphabetical order, and their total ordered amount. Customers with no orders have a total of 0 displayed due to the `COALESCE` function.



Claude does not have the ability to run the code it generates yet.

RESOURCE #6

P Write a query to:

Display the names of all customers who bought products from agents based in Chilliwack along with the names of the products they ordered, and the names of the agents who sold it to them.

Also provide what the output of the query would be on the sample data

To display the names of all customers who bought products from agents based in Chilliwack, along with the names of the products they ordered and the names of the agents who sold them, we can use the following query:

sql

Copy code

```
SELECT c.firstName || ' ' || c.lastName AS customer_name,
       p.name AS product_name,
       a.firstName || ' ' || a.lastName AS agent_name
FROM Orders o
JOIN Customers cust ON o.custId = cust.pid
JOIN People c ON cust.pid = c.pid
JOIN Products p ON o.prodId = p.prodId
JOIN Agents ag ON o.agentId = ag.pid
JOIN People a ON ag.pid = a.pid
WHERE a.homeCity = 'Chilliwack';
```

Here's how the query works:

1. The `FROM` clause starts with the `Orders` table.
2. The `JOIN` clauses join the `Customers`, `People` (for customers), `Products`, `Agents`, and `People` (for agents) tables to the `Orders` table to access the required data.
3. The `WHERE` clause filters the rows where the `homeCity` of the agent is 'Chilliwack'.

4. The `SELECT` statement selects the following columns:

- `customer_name`: Concatenation of the customer's `firstName` and `lastName` from the `People` table.
- `product_name`: The `name` column from the `Products` table.
- `agent_name`: Concatenation of the agent's `firstName` and `lastName` from the `People` table.

On the sample data provided, the output of this query would be:

Copy code

customer_name	product_name	agent_name
Maynard Ferguson	Heisenberg Compensator	Bria Skonberg
Maynard Ferguson	Universal Translator	Bria Skonberg
Maynard Ferguson	Denis Wick Valve Oil	Bria Skonberg
Louis Armstrong	Heisenberg Compensator	Bria Skonberg

This output shows the names of the customers (Maynard Ferguson and Louis Armstrong) who bought products from the agent based in Chilliwack (Bria Skonberg), along with the names of the products they ordered and the agent's name.

⚠ Claude does not have the ability to run the code it generates yet.

RESOURCE #7

P Write a query to:

Display the names of all customers who bought products from agents based in Chilliwack along with the names of the products they ordered, and the names of the agents who sold it to them.

Also provide what the output of the query would be on the sample data

To display the names of all customers who bought products from agents based in Chilliwack, along with the names of the products they ordered and the names of the agents who sold them, we can use the following query:

sql

Copy code

```
SELECT c.firstName || ' ' || c.lastName AS customer_name,
       p.name AS product_name,
       a.firstName || ' ' || a.lastName AS agent_name
FROM Orders o
JOIN Customers cust ON o.custId = cust.pid
JOIN People c ON cust.pid = c.pid
JOIN Products p ON o.prodId = p.prodId
JOIN Agents ag ON o.agentId = ag.pid
JOIN People a ON ag.pid = a.pid
WHERE a.homeCity = 'Chilliwack';
```

Here's how the query works:

1. The `FROM` clause starts with the `Orders` table.
2. The `JOIN` clauses join the `Customers`, `People` (for customers), `Products`, `Agents`, and `People` (for agents) tables to the `Orders` table to access the required data.
3. The `WHERE` clause filters the rows where the `homeCity` of the agent is 'Chilliwack'.

4. The `SELECT` statement selects the following columns:

- `customer_name`: Concatenation of the customer's `firstName` and `lastName` from the `People` table.
- `product_name`: The `name` column from the `Products` table.
- `agent_name`: Concatenation of the agent's `firstName` and `lastName` from the `People` table.

On the sample data provided, the output of this query would be:

Copy code

customer_name	product_name	agent_name
Maynard Ferguson	Heisenberg Compensator	Bria Skonberg
Maynard Ferguson	Universal Translator	Bria Skonberg
Maynard Ferguson	Denis Wick Valve Oil	Bria Skonberg
Louis Armstrong	Heisenberg Compensator	Bria Skonberg

This output shows the names of the customers (Maynard Ferguson and Louis Armstrong) who bought products from the agent based in Chilliwack (Bria Skonberg), along with the names of the products they ordered and the agent's name.

⚠ Claude does not have the ability to run the code it generates yet.

RESOURCE #8



Write a query to:

Display the first and last name of all customers who are also agents.

Also provide what the output of the query would be on the sample data

To display the first and last name of all customers who are also agents, we can use a query that joins the `Customers`, `People`, and `Agents` tables on the `pid` column, and then selects the `firstName` and `lastName` columns from the `People` table.

Here's the query:

sql Copy code

```
SELECT p.firstName, p.lastName
FROM Customers c
JOIN People p ON c.pid = p.pid
JOIN Agents a ON c.pid = a.pid;
```

Here's how the query works:

1. The `FROM` clause starts with the `Customers` table.
2. The first `JOIN` clause joins the `People` table to the `Customers` table using the `pid` column to retrieve the first and last names of the customers.
3. The second `JOIN` clause joins the `Agents` table to the `Customers` table using the `pid` column. This ensures that only customers who are also agents are included in the result set.
4. The `SELECT` statement selects the `firstName` and `lastName` columns from the `People` table.

On the sample data provided, the output of this query would be:

Copy code

firstName	lastName
Louis	Armstrong

This output shows that the only customer who is also an agent is 'Louis Armstrong'.

The query works by first selecting all customers from the `Customers` table, then joining the `People` table to get the first and last names, and finally joining the `Agents` table to filter out only those customers who are also present in the `Agents` table (i.e., customers who are also agents).

If there were no customers who were also agents in the sample data, the query would return an empty result set.

Claude does not have the ability to run the code it generates yet.

Copy Retry

CLAUDE REFERENCE

<https://claude.ai/>

ARTIFICIAL INTELLIGENCE (AI)